

Business Intelligence Assessment for Flikshop
Data Model Development and Gap Analysis for Backend Infrastructure

By
UMD iConsultancy Team
Maya Patel, Illia Polishchuk, Ashley Taupyen,
Mohammed Alzaem, Angad Sharma

Client
Flikshop

Point of Contact
Marcus Bullock

December 9, 2025

Abstract

Dear Marcus,

Thank you again for your time, flexibility, and collaboration throughout the early stages of this project. As we began reviewing the initial expectations, focused largely on UI/UX refinement and dashboard visuals, we recognized that the scope did not fully align with our team's strengths in data modeling, system analysis, and backend logic. Additionally, because the original Admin Panel designs did not contain a defined backend structure or standardized variable list, it became clear that jumping into visualizations would not meaningfully support Flikshop's long-term reporting and BI goals.

After discussing these challenges together, we jointly shifted the project toward a direction that would provide significantly more value for you and Flikshop. The new focus aligns directly with the EPIC and Requirements Document: building the foundational data model, performing a gap analysis, and proposing a normalized relational SQL schema that can support accurate reporting, consistent business logic, and future scalability.

This shift allowed us to spend our time where it matters most, helping Flikshop clarify what data the Admin Panel truly needs, how each UI element maps to a backend field, and what architecture will enable reliable dashboards and future BI development.

Revised Scope & Deliverables

Based on our conversations and the EPIC, our final deliverables include:

- **Data Requirements Spreadsheet**

A complete documentation of all data fields needed across in-scope screens (Orders, Credits, Promotions, Gift Cards, Team Events, etc.), including definitions and SQL-ready data types.

- **Gap Analysis**

A systematic comparison of the Figma UI designs against actual business logic and reporting needs, identifying missing fields, inconsistencies, and areas requiring clarification.

- **Normalized SQL Schema Proposal**

A relational database model in alignment with the EPIC's technical considerations, reducing redundancy, improving data integrity, and supporting future reporting pipelines.

- **Architecture Recommendations**

Guidance on how this data model can support Flikshop's long-term BI goals, including scalability for dashboards, automated reporting, and integration with other Flikshop services.

Each deliverable directly supports the business value outlined in the EPIC: enabling Flikshop to transition from UI concepts to a reliable, accurate, and scalable backend architecture. By establishing this foundation now, the Admin Panel, and all future analytics work, can be developed with consistency, clarity, and confidence.

We appreciate your trust in allowing us to redefine the project in a way that best supports Flikshop's vision. We look forward to presenting our final work and contributing to the long-term success of your platform.

Respectfully,

UMD iConsultancy Team

Methods

Dear Marcus,

It was a pleasure to work with you throughout this whole process and as we began working with the Flikshop Admin Panel designs, we first conducted a comprehensive review of the existing Figma files to understand the intent behind each screen and how the Admin Panel is expected to be used. Through this process, we found that while some surface-level UX and visual refinements could be suggested, the more impactful opportunities lay in clarifying the underlying data structure needed to support meaningful reporting and reliable system behavior.

Because many Admin Panel components rely on backend logic that was not yet fully defined, we recognized that a traditional gap analysis focused solely on UI discrepancies would not provide long-term value on its own. We adjusted our approach to prioritize defining data requirements and building a scalable backend foundation that could support both current needs and future growth.

To accomplish this, we reviewed each in-scope Admin Panel screen in detail and documented the data required to generate and support its functionality. This screen-by-screen analysis ensured that every UI element could be traced to a clear backend data field, improving transparency and reducing ambiguity for future development. Using this information, we organized and normalized the data from First Normal Form (1NF) through Third Normal Form (3NF), ensuring consistency while avoiding unnecessary duplication.

Building on this work, we designed a relational SQL schema that clearly defines how information is connected across users, orders, events, and administrative processes. This structure provides a strong foundation for accurate reporting, dashboard development, and long-term analytics, while also enabling Flikshop's technical team to integrate and extend the system more easily in the future.

To further support usability and adoption, we created sample data aligned with the finalized schema. This helps illustrate how data flows through the system and makes it easier for engineers, analysts, and stakeholders to understand how the model functions in practice. Together, this approach bridges the gap between visual design and backend implementation, ensuring that Flikshop's Admin Panel can scale with confidence as its data and reporting needs evolve.

Respectfully,

UMD iConsultancy Team

Deliverables, Findings, and Maintenance Documentation

Data Catalog

The data catalog is a comprehensive reference document that breaks down every component of your database structure in a clear, accessible format. For each of the 34 tables in the normalized database, the catalog provides variable names and their purpose, data types that are SQL compatible, example values showing real world usage, constraints such as unique values and null permissions, preset value indicators for fields that should use dropdown or lookup tables, and cross references to lookup tables for standardized values.

The data catalog serves as the "dictionary" for your database, making complex technical structures accessible to both technical and nontechnical staff. Rather than deciphering an ERD diagram, team members can quickly understand what data exists, how it is structured, and what values are acceptable.

For the Technical Team: Use the data catalog as your primary reference during database implementation. Validate that all constraints documented in the catalog are properly enforced in your database schema. Implement foreign key relationships as specified in the cross reference section. Reference the catalog when writing queries to ensure proper table joins and field selection.

Sample Data Documentation

The sample data package provides realistic example records for all 34 normalized tables, demonstrating how data flows through your system. Unlike randomly generated data, these examples are strategically designed to illustrate how primary keys uniquely identify records within their source tables, how foreign keys create relationships between tables, real world scenarios such as a user sending a postcard to a recipient at a specific facility, and more.

Sample data transforms abstract database concepts into concrete, understandable examples. This is particularly valuable because nontechnical staff can see exactly what the database will contain and how it relates to business operations. The technical team can test database functionality with realistic data before going to production. Customer support and operations teams can understand data relationships without technical training. Quality assurance teams can use the sample data as a testing baseline. The sample data demonstrates the business logic your system needs to support, such as one facility housing multiple recipients, with each recipient receiving multiple orders.

For the Technical Team: Replace generic placeholder values such as "Prison Name 1" or "Long description 3" with production-quality sample data that reflects your actual business context. Maintain the same number of rows per table to preserve referential integrity. Save each table as a

separate CSV or Excel file for SQL import. To forward engineer the database, open MySQL Workbench and load the provided ERD diagram file. Navigate to Database and select Forward Engineer to create the schema structure. Verify that all 34 tables, foreign keys, and constraints were created successfully. Import lookup and reference tables first, as other tables depend on these. Import core tables in dependency order: Address, then User, then Orders, followed by related tables. Use MySQL's Table Data Import Wizard or LOAD DATA INFILE commands for bulk import. Verify foreign key relationships by running test queries that join related tables. Check data integrity with COUNT queries to ensure all records imported successfully. Remember that import order matters due to foreign key constraints. Always import parent tables, those referenced by foreign keys, before child tables, those containing foreign keys.

SQL Entity Relationship Diagram

The ERD diagram is a visual blueprint of your entire database architecture, showing all 34 tables, their relationships, and data dependencies. Created in MySQL Workbench format, this diagram maps all tables with their complete field structures, primary keys as unique identifiers for each record, foreign keys as links between related tables, data types and constraints for each field, one-to-many and many-to-many relationships, and the normalized structure that eliminates data redundancy.

The ERD diagram is the architectural foundation of your data infrastructure and provides critical value in several ways. The MySQL format allows your technical team to generate the database structure automatically, significantly reducing development time. The normalized design using Third Normal Form provides a clean foundation for adding new features without restructuring data. The visual representation helps staff of different disciplines discuss data requirements using a common reference. The diagram clearly shows what data relationships must work correctly, making testing more systematic.

For the Technical Team: Open the provided .mwb file in MySQL Workbench. Review the entire schema to understand table relationships and dependencies. Use Database then Forward Engineer to generate SQL CREATE statements. Select your target MySQL server and database name. Review the generated SQL script before execution to ensure it matches your environment requirements. Execute the forward engineering process to create all tables, indexes, and constraints. Verify the database structure using SHOW TABLES and DESCRIBE commands. Test foreign key relationships with sample INSERT statements. Keep the .mwb file under version control as your authoritative database schema definition. Use Database then Reverse Engineer on your production database periodically to update the diagram if schema changes are made.

Data Visualization Query Documentation

This deliverable provides detailed pseudocode and step-by-step query logic for creating some of key data visualizations from your Figma designs. Each visualization includes a visual reference showing the exact chart or table from Figma, bullet-point pseudocode explaining the query logic in layman's terms, identification of which tables and fields are required, step-by-step breakdown of joins, filters, aggregations, and calculations, notes on handling edge cases and data quality issues, and output format specifications for frontend integration.

Creating accurate queries for data visualizations is often more complex than it appears. This documentation provides critical value by bridging design and development, translating beautiful Figma designs into concrete technical requirements and eliminating ambiguity about what data each visualization needs. It accelerates development because your technical team does not need to reverse-engineer the business logic, as it is already documented in clear, implementable steps. The pseudocode is based on your actual normalized database structure, ensuring queries will work with your data relationships and guaranteeing accuracy. As your database schema evolves, the logical approach documented here can be adapted rather than starting from scratch, providing future flexibility. The documentation highlights where indexes, aggregations, and joins are needed, helping your team write efficient queries from the start and enabling performance optimization.

For the Technical Team: Complete the forward engineering process using the ERD diagram and import the sample data. Verify all tables and relationships are functioning correctly. For each visualization, review the pseudocode and understand the business logic. Identify the source tables and required joins. Start with a simple SELECT statement and add complexity incrementally. Test each JOIN, WHERE clause, and aggregation separately. Verify your results against the sample data to ensure accuracy. Add appropriate indexes on columns used in WHERE, JOIN, and ORDER BY clauses. For faster implementation, you can use AI tools to generate SQL from the pseudocode. Provide the AI with your ERD structure including table names, fields, and relationships. Share the specific pseudocode for the visualization you are building. Request SQL query generation with your specific SQL dialect, MySQL. Review and test the generated code, validating AI output with your sample data. Rather than running complex queries directly from your application, create SQL views for each visualization. Views simplify application code, improve maintainability, provide a consistent data access layer, and can be optimized independently. Run EXPLAIN on each query to understand execution plans. Add indexes on foreign keys and frequently filtered columns. Consider materialized views for complex aggregations that do not need real-time data. Implement caching for dashboards that refresh periodically rather than on every page load. Monitor query performance in production and optimize slow queries. We intentionally provided pseudocode rather than final SQL because your team may modify the database architecture based on performance needs or feature

requirements. The logical approach documented in the pseudocode will remain valid even if table structures evolve.

Gap Analysis and Recommendations

The gap analysis provides a comprehensive evaluation of your current website architecture and database design, identifying critical areas for improvement. Through detailed analysis of your existing system, business requirements, and industry best practices, we have identified five key areas that require attention before scaling your platform. Each identified gap includes a detailed description of the current limitation or issue and specific actionable recommendations for resolution.

The gap analysis represents preventive maintenance for your platform. Addressing these issues now, before they cause problems, is significantly more cost-effective than fixing them after launch or during rapid growth. The value includes risk mitigation by identifying potential data inconsistencies before they impact the development. It provides cost savings because fixing architectural issues before production deployment costs a fraction of what emergency fixes cost during active operations. The analysis ensures scalability assurance by confirming your database can handle growth from hundreds to thousands or even millions of users without requiring expensive re-architecture. It creates competitive advantage because a solid technical foundation allows you to iterate faster, launch features more quickly, and provide better user experience than competitors. Finally, it builds investor confidence by demonstrating technical due diligence and reducing perceived risk for potential investors or partners.

For the Technical Team: Review each identified gap with both technical and business stakeholders to understand the full context. Prioritize gaps based on risk to business operations, complexity of implementation, and dependencies on other fixes. Create detailed technical specifications for each recommended solution. Estimate technical effort and create an implementation timeline. Implement fixes in a development environment and thoroughly test before production deployment. Update all documentation including the ERD and data catalog to reflect changes. Monitor system performance and data integrity after each fix is deployed. Schedule regular architecture reviews, quarterly is recommended, to identify new gaps as the platform evolves. Many organizations postpone addressing architectural gaps due to immediate feature demands. However, technical debt compounds quickly. We strongly recommend addressing all high-priority gaps identified in the analysis before major feature releases or marketing campaigns.

Deliverables Summary

These five deliverables work together as a comprehensive database implementation package. The Data Catalog provides the reference guide. The Sample Data demonstrates real-world usage. The

ERD Diagram defines the architecture. The Query Documentation enables visualization implementation. The Gap Analysis ensures long-term success. Your technical team has everything needed to implement a production-ready, scalable database that supports your current business needs while providing flexibility for future growth.

Recommendations

Through a comprehensive analysis of Flikshop's current database architecture and user interface design, our team has identified five critical areas for improvement that will enhance data integrity, improve user experience, and ensure scalability as the platform grows. These recommendations address structural issues in the database schema, user interface inconsistencies, and missing data capture points that could limit functionality or create data quality problems. Each recommendation has been prioritized based on its impact on system reliability, user experience, and implementation complexity.

Collect Phone Number and Location During Account Setup

Currently, user phone numbers and address information are optional fields that can be added later in the user profile. However, these data points are critical for core business operations including order fulfillment, customer support, and geographic analytics displayed in your dashboard visualizations. We recommend making phone number and location mandatory fields during the initial account setup process. This ensures complete user profiles from the start, eliminates data gaps that complicate operations, and enables immediate access to location-based features and analytics. The address collection should capture the full geographic hierarchy already designed in your database: address line 1, address line 2, city, state, and zipcode. This approach prevents incomplete records that require follow-up data collection and supports the geographic visualizations in your dashboard that segment users by state and country. From a technical perspective, implementing this change requires updating the account creation form to include required address fields and phone number validation, adding client-side and server-side validation to ensure data completeness before account creation, and updating the User table constraints to mark phone_number and address_id as NOT NULL fields.

Implement Role as a Lookup Table with Dropdown Selection

The current database structure includes a role field in the Admin table, but role is stored as a text field rather than referencing a standardized lookup table. This structure should be reflected in the user interface through dropdown menus rather than free-text entry. Using a dropdown menu for role selection prevents data inconsistencies caused by typos or variations in role naming, ensures that only valid, predefined roles can be assigned, simplifies role-based access control implementation, and makes it easier to update role definitions system-wide when business needs change. This recommendation applies to all fields in your database that reference lookup tables, including customer type, order type, order status, tag names, and request reasons. Any field with a finite set of possible values should use a dropdown interface that pulls from the corresponding lookup table. Implementation requires updating admin creation and editing forms to replace text inputs with dropdown menus populated from the Admin_Role table, applying the same pattern to

all other fields that reference lookup tables throughout the application, and ensuring the frontend enforces referential integrity by only allowing selection of valid foreign key values.

Standardize All Finite-Value Fields with Dropdown Menus

Building on the role recommendation, we identified multiple fields throughout your application that should consistently use dropdown menus because they reference lookup tables with finite options. These include customer type with options like SUPER USER, RISING STAR, and FLIKSHOP ANGEL, order type including postcard, print, and flikbook, tag names used for categorizing users and groups, order status values such as delivered, processed, and pending, event status options, request status values, and request reasons for credit requests. Inconsistent implementation of dropdowns versus free-text entry creates data quality problems, makes reporting unreliable, and increases the likelihood of invalid foreign key references that cause application errors. By standardizing the user interface to use dropdowns for all lookup table references, you ensure data consistency, improve user experience by showing available options clearly, prevent database errors from invalid foreign key values, and make the application more maintainable as business rules evolve. From a technical implementation standpoint, the tech team should create reusable dropdown components in the frontend framework that automatically populate from lookup tables, implement a consistent pattern where any field with a foreign key relationship to a lookup table uses a dropdown, add validation to reject manual entry or invalid selections, and ensure the application handles cases where lookup tables are empty or unavailable gracefully.

Add Recipient Field to Notes Table

The Notes table currently captures which user created a note and includes general subject and content fields, but it lacks a clear indication of who the note is directed toward or meant for. Without a recipient field, notes lack crucial context about their purpose and intended audience. This enhancement provides clear context for every note, enables filtering and querying of notes by intended recipient, supports customer service workflows by showing which notes relate to which recipients, and improves audit trails for compliance and quality assurance purposes.

Display End Dates for Team Events

The Team Events table in your database includes both `event_start_date` and `event_end_date` fields, indicating that events can span multiple days or have defined completion dates. However, the current user interface may not consistently display the end date, showing only start dates or displaying incomplete event information. For events that span multiple days or have specific deadlines, showing only the start date creates ambiguity about event duration and completion status. Users cannot easily determine if an event is ongoing, completed, or upcoming without seeing the end date. We recommend updating all Team Event displays to show both start and end

dates clearly. This improvement enhances user understanding of event timelines, supports better planning and scheduling, provides complete information for event tracking and reporting, and aligns the user interface with the data structure already designed in your database.

Implementation requires updating event display components to show both start and end dates, adding appropriate formatting logic for date ranges, ensuring null end dates are handled gracefully with appropriate messaging, and verifying that event listings, calendars, and detail views all display complete date information consistently.

Implementation Priority

These five recommendations are listed in order of implementation priority based on their impact on data quality and system reliability. The phone number and location collection during account setup should be implemented first as it affects every new user and prevents data gaps that are difficult to fill retroactively. The role and dropdown standardization recommendations should follow as they prevent data inconsistencies that compound over time. The notes recipient field and team event end date display are important for user experience but have less impact on core system functionality. All five recommendations can be implemented incrementally without requiring major architectural changes, as the underlying database structure already supports these features through the normalized design we have developed.

Conclusions

Project Redirection and Focus Shift

This report outlines the successful redirection of the initial project scope to better align with Flikshop's long-term business intelligence (BI) and reporting objectives. After evaluating the challenges in the original plan, which focused primarily on UI/UX refinement and dashboard visuals, it became clear that the project's impact could be maximized by addressing the core issues in the backend architecture and data modeling. The decision to shift the project's focus toward building a strong data foundation was made collaboratively, ensuring that Flikshop would have a reliable and scalable backend system that could support both current and future BI needs.

Key Deliverables and Outcomes

The final deliverables—comprising a data requirements spreadsheet, a detailed gap analysis, a normalized SQL schema proposal, and architecture recommendations—have provided Flikshop with a comprehensive roadmap to move forward with confidence. The data requirements documentation ensures that all in-scope screens are accurately captured, while the gap analysis has identified and addressed inconsistencies and areas for improvement in the existing system. The normalized SQL schema proposal offers a structure that reduces data redundancy and supports long-term scalability, while the architecture recommendations outline how this data model can evolve to accommodate future BI initiatives.

Gap Analysis and Improvement Recommendations

Furthermore, the gap analysis has highlighted five key areas for improvement, ranging from enhancing data collection during user account setup to standardizing the use of dropdown menus for role-based and lookup table fields. By prioritizing these improvements based on their impact on data quality and user experience, this report ensures that Flikshop is not only addressing immediate concerns but also preparing for future growth. The implementation of these recommendations will significantly reduce the risk of data inconsistencies and improve the overall user experience, making the platform more reliable, efficient, and scalable.

Long-Term Impact and Scalability

In conclusion, this project has laid a solid foundation for Flikshop's transition from visual mockups and UI concepts to a fully functional, data-driven backend infrastructure. The work completed here has positioned the company to move forward with a clearer understanding of its data needs and how best to meet them. As Flikshop continues to scale and evolve, the tools and recommendations provided will serve as a critical asset in ensuring that future analytics and reporting systems can be developed with consistency, clarity, and confidence.

Attachments and Links

Here is the link to the Google Drive containing all of the deliverables:

 [Final Deliverables to Give to Marcus](#)

The deliverables will also be emailed to you.